

PA 1: Implementing xv6 User Commands and System Calls

This report describes the xv6 utilities, CPU accounting, and wait2 system call implemented on the pa1-syscalls branch. The observed tests distinguish elapsed time from CPU time: sleep used 0 CPU ticks, one matrix multiplication used 100% CPU, and two simultaneous runs each used about half of the single CPU.

Task 1. (5 pts) Implement the uptime utility

I added user/uptime.c and the `_uptime` entry in the Makefile UPROGS list. The utility calls the `uptime` system call to report the number of timer ticks since xv6 booted. Test command: `uptime`. Observed output: [paste the actual output from your xv6 terminal here]. The tick count depends on when the command is run.

```
init: starting sh
$ uptime
up 1360 clock ticks
$ QEMU: Terminated
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

I added user/time1.c and `_time1` to UPROGS. The program records uptime, forks, runs the requested command with `exec` in the child, waits for it in the parent, and subtracts the starting tick count from the ending count.
Test: `time1 sleep 20`
Observed output: elapsed time: 20 ticks. This confirms that `time1` measures wall clock time while the child sleeps.

```
init: starting sh
$
$ time1 sleep 20
elapsed time: 20 ticks
$
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

In kernel/proc.h I added the `cputime` field to `struct proc`. In kernel/proc.c I initialized and reset it as processes are allocated and freed. In kernel/trap.c I incremented it when a process exhausted a timer interval in `usertrap` or `kerneltrap`, under the process lock before yield. These changes account for CPU ticks used by the scheduled process. Test evidence: `time1 sleep 20` measured 20 elapsed ticks; later the time utility reported 0 CPU ticks for sleep 10.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

I added kernel/pstat.h with `struct rusage` containing `uint cputime`. In user/user.h I declared `struct rusage` and the `wait2` prototype; user/usys.pl generates the user stub. In kernel/syscall.h and kernel/syscall.c I assigned system call number 23 and added its dispatcher entry. In kernel/sysproc.c `sys_wait2` reads two user addresses and calls `kwait2`, whose prototype is in kernel/defs.h. In kernel/proc.c `kwait2` finds an exited child, copies its exit status and `cputime` to the parent using `copyout`, and reaps the child. The `copyout` call uses this xv6 version's five

arguments, including the parent process size. The successful time sleep 10 and time matmul tests exercise wait2 and its rusage result.

```
int
kwait2(uint64 addr, uint64 usage_addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();
    struct rusage usage;

    acquire(&wait_lock);
    for (;;) {
        havekids = 0;
        for (pp = proc; pp < &proc[NPROC]; pp++) {
            if (pp->parent == p) {
                acquire(&pp->lock);
                havekids = 1;
                if (pp->state == ZOMBIE) {
                    pid = pp->pid;
                    usage.cputime = pp->cputime;

                    if (addr != 0 &&
                        copyout(p->pagetable, p->sz, addr, (char *)&pp->xstate,
                               sizeof(pp->xstate)) < 0) {
                        release(&pp->lock);
                        release(&wait_lock);
                        return -1;
                    }
                }
                if (usage_addr != 0 &&
                    copyout(p->pagetable, p->sz, usage_addr, (char *)&usage,
                           sizeof(usage)) < 0) {
                    release(&pp->lock);
                    release(&wait_lock);
                    return -1;
                }
            }
        }
    }
}
```

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

I added user/time.c and _time to UPROGS in the Makefile. The parent records uptime, forks a child to exec the requested command, calls wait2 for the exit status and rusage, and computes CPU percentage as $100 \times \text{cputime} / \text{elapsed ticks}$. When elapsed time is zero, it reports 0% to avoid division by zero. I also added user/matmul.c as a CPU intensive workload and used the existing sleep command for an idle workload.

Observed tests in xv6:

time sleep 10 → elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU

time matmul → elapsed time: 14 ticks, cpu time: 14 ticks, 100% CPU

time matmul & time matmul & → first: 25 elapsed, 12 CPU ticks, 48% CPU; second: 24 elapsed, 13 CPU ticks, 54% CPU.

The parallel results show that the two processes share the single configured CPU. Their exact tick counts may vary between runs.

```
init: starting sh
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$

init: starting sh
$ time matmul
Time: 14 ticks, checksum: 1247680
elapsed time: 14 ticks, cpu time: 14 ticks, 100% CPU
$ time matmul & time matmul &
$ Time: 23 ticks, checksum: 1247680
elapsed time: 25 ticks, cpu time: 12 ticks, 48% CPU
Time: 23 ticks, checksum: 1247680
elapsed time: 24 ticks, cpu time: 13 ticks, 54% CPU
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

I learned how user command stubs, system call numbers and dispatch, kernel implementations, process structures, and copyout connect across xv6. The measurements made the difference between elapsed time and CPU time concrete. My first kernel/trap.c edit failed to compile because a stray character changed the syntax; I corrected the function and rebuilt. I also initially called copyout with the wrong argument list, then checked kernel/defs.h and supplied the process size required by this xv6 version. I used git diff --check to remove trailing whitespace and verified the final build and tests before committing. The code was committed on pa1-syscalls as 9a38975 for CPU tracking and 57ab229 for wait2 and time.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

The rusage returned by wait2 contains CPU time only for its direct child. If the timed command forks workers, their CPU time is not included in the printed total, even though the parent command may wait for them. Consequently, the displayed %CPU can understate the CPU used by the whole job. This version also reports whole timer ticks, so short commands may show zero elapsed ticks and a coarse or misleading percentage. It uses one child and does not aggregate pipelines or multiple shell commands.

Use of AI assistance

I used OpenAI ChatGPT/Codex when I had questions about the xv6 tasks or encountered errors in my code. It helped me understand compiler messages and possible corrections, which I reviewed and tested in my Ubuntu VM.